CSC209 – Final Project Proposal

Sylvia Wong, Raiyan Tanvir Ta-seen, Yifan He

Description of Program

We will design a concurrent C program to process .bmp image files through several layers of image filtering such as grayscale, gaussian blur, sepia, colour inversion, etc. Image file names will be specified through command line arguments, along with the order of image filters they want to apply. We will start by implementing three filters in the pipeline. Usage may look like: ./program [filename] [n number of filters] [filter 1] [filter 2][filter n]. This is an interesting application because it would allow us to handle applying multiple image filters to a single image faster using concurrency, rather than applying one filter at a time.

Sketch of Communication Protocols

Starting from the parent process, we will want to first analyze the command line arguments provided by the user. Then, we load the image using standard C techniques for loading .bmp files as we have done in Lab 6. From the parent, we will use *fork* to create a child process for each filter in the pipeline and *pipes* will be created between each child in the pipeline, so the output of one child is the input to the next. The parent will then divide the image into subsections and write each subsection to the first child in the pipeline. Then each child process will apply its filter to the row and write the processed row to the next child through pipes. In the case of complex filters like gaussian blur, we may want to consider a different structure to designate each child to work on, perhaps a matrix of pixels instead of rows. In this case, we can use our new structure, using the C struct, to designate the working space for a child to avoid any conflicts in complex image processing requiring neighbouring pixels. The struct can be modified per child to indicate the size of the matrix it is working on, which is crucial for gaussian blur since at the boundaries, it needs neighbouring pixels, which may have not been processed by the last filter yet. The child processes handling the last filter will communicate the output image data to the parent using pipes, who will reassemble the image file given the position of the subsection and produce a new image file called [filename]_[filter 1]_[filter 2]_..._[filter n].bmp. Each child process will work on a subsection of image data, and move on to the next, limited by how fast the previous child process is. Given that each child works relatively fast on a filter in that chunk, the processing will become extremely fast and marginally better than applying filters in a linear sequence.

Our current list of filters we want work on: Grayscale, Sharpen, Emboss, Gaussian Blur, Edge Detection using Sobel, Glitch Effect, Sepia, Invert, Brightness Adjustment, Contrast Adjustment, Black-and-White, Pixelate, Posterize, Vignette

Note: We are not sure which filters to tackle since some are harder to implement and how many filters we should have per person. Could we please get some feedback on this? Thank you.